

AHMADU BELLO UNIVERSITY, ZARIA

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COEN838: ADVANCED SOFTWARE ENGINEERING

Project Examination, Question 4

OFFLINE-FIRST CRYPTOGRAPHIC QR IDENTITY WALLET

FOR RURAL PUBLIC SERVICES

TEST PLAN AND TEST REPORT

Document QRW-TST-04 · Version 1.0

BY

ALIYU SARKI ILIYASU

P25EGCP8040

November 2026

DOCUMENT CONTROL

Document details

Item	Detail
Document	Test Plan and Test Report (QRW-TST-04)
Project	Offline-First Cryptographic QR Identity Wallet for Rural Public Services
Course	COEN838 Advanced Software Engineering, Ahmadu Bello University, Zaria
Author	Aliyu Sarki Iliyasu (P25EGCP8040)
Version / date	1.0 · November 2026
Source code	https://github.com/sarki94/qr-identity-wallet
Related documents	QRW-SRS-01 Requirements · QRW-SDD-02 Design · QRW-SEC-03 Security

Revision history

Version	Date	Change
1.0	4 Nov 2026	First complete test plan and test report

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Purpose....1
1.2 Scope....1
1.3 Summary of Results....1
1.4 References....1
2. TEST STRATEGY....3
2.1 Overview....3
2.2 Test-Driven Development....4
2.3 Property-Based Testing....4
2.4 Cross-Implementation Testing....5
2.5 Requirements-Based Test Design and Traceability....5
2.6 Continuous Integration....6
3. TEST PLAN....7
3.1 Test Items....7
3.2 Features to Be Tested....7
3.3 Features Not Tested, and Why....7
3.4 Pass and Fail Criteria....7
3.5 Suspension and Resumption....8
3.6 Test Environment....8
3.7 Test Data....8
3.8 Responsibilities and Schedule....9
3.9 Risks and Contingencies....9
4. TEST CASES....10
4.1 Core Library....10
4.2 Services....11
4.3 Security....12
4.4 Browser Library....12
4.5 System Tests in the Browser....13
5. TEST RESULTS....14
5.1 Results by Level....14
5.2 Code Coverage....14
5.3 Requirements Coverage....15
5.4 Performance and Capacity....16
5.5 System Test Results....17
5.6 Static Analysis, Security Scans and Continuous Integration....17

6. DEFECTS FOUND..... 19
7. EVALUATION AND CONCLUSION..... 20
7.1 Evaluation..... 20
7.2 Limitations..... 20
7.3 Conclusion..... 20

1. INTRODUCTION

1.1 Purpose

This document is both the test plan and the test report for the Offline-First Cryptographic QR Identity Wallet. Part one (Sections 2 and 3) states the test strategy and the plan: what is tested, at which levels, with which techniques and tools, and what counts as a pass. Part two (Sections 4 to 7) lists the test cases, reports the results, coverage and measurements, records the defects the tests found, and evaluates how far the evidence supports the claim that the system meets its requirements. The structure follows the test plan and test summary report of IEEE 829 (IEEE, 2008).

1.2 Scope

All three code bases are in scope: the cryptographic core library (qrw_core), the five back-end services (qrw_services) and the web application's verification library and screens. Security testing is summarised here and reported in full in QRW-SEC-03.

1.3 Summary of Results

94 test cases were designed, executing **145 individual checks** (some cases are parameterised or run several steps). **All 94 pass.** Statement coverage is 97% for the core library and 97% for the services; the browser verification library is at 90% and the PIN vault at 100%. Every one of the 48 requirements is linked to at least one passing test or a recorded manual verification, and no requirement is uncovered. Automated tests, scans and the candidate's own manual testing found seven real defects during development, all fixed (Section 6).

1.4 References

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.

Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, 268–279. https://doi.org/10.1145/351240.351266

IEEE. (2008). *IEEE standard for software and system test documentation* (IEEE Std 829-2008). Institute of Electrical and Electronics Engineers.

MacIver, D. R., Hatfield-Dodds, Z., & Many Other Contributors. (2019). Hypothesis: A new approach to property-based testing. *Journal of Open Source Software, 4*(43), 1891. https://doi.org/10.21105/joss.01891

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

2. TEST STRATEGY

2.1 Overview

The strategy follows the risk of each part of the system. The trust decision (signatures, challenge–response, revocation and the audit chain) is where a defect would let a fraud through, so it gets the strongest techniques: test-driven development, property-based testing and cross-implementation testing between Python and the browser. The services are tested through their public APIs, black-box. The whole system is tested end to end both through the APIs and through real browsers. Security, performance and static analysis run alongside, and continuous integration runs everything that can be automated on every push.

Table 2.1 shows the levels, their tools and the number of test cases at each level.

Table 2.1: Test levels, tools and counts

Level	What is tested	Tools	Test cases	Checks run
Unit (core library)	Signing, verification, encodings, QR format, challenge, revocation list, hash chain	pytest, Hypothesis	38	50
Unit, performance (core)	Verification time, list size, signing throughput	pytest, Hypothesis	3	3
Unit (browser library)	Browser verifier against Python-issued vectors; PIN vault; cache	Vitest	16	16
Integration (service APIs)	Each service's API with real databases and keys, wired together	pytest, FastAPI TestClient	19	21
Integration, performance and capacity	Issuance time, batch upload, compressed list	pytest, FastAPI TestClient	3	3
Security (black-box)	Injection, token forgery, access control, headers, CORS, errors	pytest, FastAPI TestClient	12	24
System (API, end to end)	A full distribution day through every service	pytest, FastAPI TestClient	1	1
System (browser, Playwright)	The real PWA on three browser devices; the 16-step Guided Demo	Playwright	2	27
Total			94	145

In addition, static analysis (Flake8, Bandit, TypeScript strict type-checking), dependency scanning (pip-audit, npm audit) and a dynamic scan with OWASP ZAP were run; they are reported in QRW-SEC-03 and summarised in Section 5.6.

2.2 Test-Driven Development

The core library was built test-first in the red–green–refactor cycle (Beck, 2003). The complete unit test suite for the library was written and committed before any implementation existed, and the commit history shows the failing suite followed by the implementation that makes it pass. The same discipline was used later for the defect found by the ZAP scan. Because the history is in Git, the order can be checked by anyone (Table 2.2).

Table 2.2: Test-first evidence in the commit history

Commit	Date	Step	Message
1411e75	30 Oct 2026	Red	test(core): add failing test suite for qrw_core (TDD red)
0c0673a	30 Oct 2026	Green	feat(core): implement qrw_core to pass test suite (TDD green)
36b9f76	30 Oct 2026	Refactor	test(core): cover key storage and validation branches; add CI pipeline
916f0ad	4 Nov 2026	Red	test(security): TC-SEC-011/012 for the ZAP finding (red)
7c44f43	4 Nov 2026	Green	fix(security): strict CSP on API docs pages; ZAP scan evidence (green)

Behaviour-driven development (BDD) was considered and not adopted. Its main benefit is a shared, readable specification between developers and non-technical stakeholders; in a solo project that role is already played by the requirements in QRW-SRS-01, which every test names through its requirement tag. Writing Gherkin scenarios as well would have duplicated that specification without adding checks. The two system tests do, however, follow scenario form (given an issued credential, when the agent scans offline, then the verdict is accepted), which captures the main benefit of BDD where it matters.

2.3 Property-Based Testing

For the security-critical functions, example tests alone are weak: they only check the cases the developer thought of. Property-based tests state a rule that must hold for every input and let Hypothesis, the Python counterpart of QuickCheck (Claessen & Hughes, 2000), generate a hundred or more inputs per test run, including edge cases, and shrink any failure to the smallest example. 7 test cases use this technique (Table 2.3).

Table 2.3: Properties tested with Hypothesis

Test case	Property that must hold for all inputs
TC-CORE-003	Base45 round trip property
TC-CORE-005	Base64url round trip property
TC-CORE-014	Any single bit flip is rejected
TC-CORE-019	Garbage never raises and is rejected
TC-CORE-020	Any valid text round trips
TC-CORE-023	Arbitrary text only raises invalid QR
TC-CORE-051	Editing any record is detected

The most important is TC-CORE-014: for a valid credential, flipping any single bit anywhere in the signature or payload must lead to rejection. It is the executable form of the integrity requirement.

2.4 Cross-Implementation Testing

Credentials are issued by Python on the server and verified by TypeScript in the browser. Any difference between the two (in canonical bytes, encoding or the Ed25519 verification rules) would cause genuine farmers to be rejected or, worse, forged credentials to be accepted. A script in the core library generates test vectors with the Python implementation (valid, tampered, revoked, expired and foreign-issuer credentials, a signed revocation list and challenge answers), and the browser tests must reach the same verdicts. TC-WEB-006 goes further: Ed25519 signatures are deterministic, so the browser must produce byte-for-byte the same challenge answer as Python.

2.5 Requirements-Based Test Design and Traceability

Every test names the requirements it verifies: Python tests with a marker, for example @pytest.mark.req("FR-16", "NFR-S5"), and browser tests in their title, for example "TC-WEB-003 [FR-16, NFR-S5]". A script reads these tags and the JUnit results and generates the Requirements Traceability Matrix, so the RTM always reflects the tests that actually ran and whether they passed. A requirement with no passing test shows up as uncovered. Test cases were designed from the requirements with equivalence partitioning (valid, tampered, expired, revoked, unknown issuer, malformed) and boundary values (list age at 7 and 30 days, a 2,000-record batch, an 11-digit NIN, the 8 KB inflate limit).

2.6 Continuous Integration

Every push to GitHub runs the pipeline in Table 2.4. A job fails, and the commit is marked red, if any test fails, coverage falls below 90%, the linters or Bandit report an issue, or a shipped dependency has a known moderate or higher vulnerability.

Table 2.4: Continuous integration pipeline (GitHub Actions)

Job	Steps	Quality gate
core	Install; Flake8; Bandit; pytest with Hypothesis and coverage; upload JUnit and coverage	All tests pass; coverage ≥ 90%; no lint or Bandit issues
services	Install; Flake8; Bandit; pip-audit; pytest (integration, security, end to end) with coverage	All tests pass; coverage ≥ 90%; no known vulnerable dependency
web	npm ci; npm audit of shipped packages; TypeScript strict type-check; Vitest with coverage; production build	Type-check clean; all tests pass; no moderate+ advisory; build succeeds
docker	Build the service image	Image builds

3. TEST PLAN

3.1 Test Items

Table 3.1: Test items

Item	Version / location
qrw_core cryptographic library	core/qrw_core, Python package 0.1.0
Auth, Issuer, Trust and Revocation, Audit and Sync, Mock NIMC services	services/qrw_services, FastAPI 0.162
Web application (Wallet, Verifier, Issuer Portal, Audit, Guided Demo)	web/src, React 18.3 + TypeScript 5.6, Vite 5.4
Docker image and Compose configuration	Dockerfile, docker-compose.yml

3.2 Features to Be Tested

All 29 functional and 19 non-functional requirements of QRW-SRS-01: enrolment and issuance, holder binding, offline verification at both assurance levels, revocation and its distribution, synchronisation and the audit chain, staff authentication and roles, and the performance, security, capacity and usability requirements.

3.3 Features Not Tested, and Why

- **Real NIMC integration.** The NIMC interface is not available to students; a mock with fictional records is used (assumption in QRW-SRS-01).
- **Field use on low-end Android phones in Kura.** Planned with npm run phone on the candidate's own phone before the defence; not yet done at the time of writing. Performance was measured in a test environment, with large margins (Section 5.4).
- **Load from many concurrent users.** Verification runs on the phones, so server load comes only from synchronisation, which is batched; a load test was judged lower value than the security tests for the time available.
- **Deferred features** (staff multi-factor authentication, full Hausa translation, agent PIN unlock): not built, so not tested. They are the five requirements marked Partial in the RTM.

3.4 Pass and Fail Criteria

- A test case passes when every assertion holds; one failed check fails the case.

- The test run passes when all automated test cases pass, statement coverage is at least 90% for the core library and the services, and static analysis and dependency scans report no unresolved issue.
- The release is acceptable when every Must requirement is covered by at least one passing test or a recorded manual verification, and no High or Medium finding from the dynamic scan is open.

3.5 Suspension and Resumption

Testing of a component is suspended when its tests cannot run (for example, a service fails to start or the vectors file is missing), because results would be meaningless. It resumes when the cause is fixed; the full suite is then re-run, not only the failed tests.

3.6 Test Environment

Table 3.2: Test environment

Element	Development and test	Continuous integration
Operating system	Windows laptop (candidate); Linux container	Ubuntu (GitHub-hosted runner)
Python and test tools	Python 3.13; pytest 9.2; Hypothesis 6.170; coverage.py	Python 3.12, same tools
Node and test tools	Node 22; Vitest 2.1.4 with V8 coverage; Playwright with Chromium	Node 20; Vitest
Services	Five services on localhost (8001 to 8005), SQLite, temporary keys per test	In-process TestClient, temporary databases
Browsers	Chromium (Playwright), three isolated browser contexts as three devices	Not run in CI
Security tools	Bandit, Flake8, pip-audit, npm audit, OWASP ZAP 2.16.1	Bandit, Flake8, pip-audit, npm audit

3.7 Test Data

All test data is fictional. The mock NIMC service holds invented NIN records for Kura LGA (for example, Musa Ibrahim, Kura Ward). Each test creates its own keys and databases in a temporary folder, so tests never share state and can run in any order. Cross-implementation vectors are generated by a script and committed, so the browser tests are repeatable.

3.8 Responsibilities and Schedule

This is a solo project: the candidate designed, wrote and ran all tests. Testing ran alongside development, as shown in Table 3.3.

Table 3.3: Testing schedule

Date	Testing activity
30 Oct 2026	Core unit and property-based tests written first (red), then implementation (green); CI set up
3 Nov 2026	Service integration and end-to-end tests; browser library tests; Playwright system tests TC-SYS-002 and TC-SYS-003; candidate's manual run of the Guided Demo on his laptop
3 Nov 2026	Security tests, static analysis, dependency audit; performance and capacity tests; generated RTM
4 Nov 2026	OWASP ZAP scan, fix and re-scan; coverage for the browser library; this report
Before 8 Nov 2026	Field-style test on a real phone over HTTPS (npm run phone)

3.9 Risks and Contingencies

Table 3.4: Testing risks

Risk	Contingency
Python and browser disagree on signature rules	Shared vectors; strict RFC 8032 mode in the browser (TC-WEB-002 to 008)
Tests pass but the user interface is wrong	Playwright system tests drive the real screens and save screenshots
Timing-based performance tests are flaky on slow machines	Budgets set far above measured values; measured figures printed
Limited time before the deadline	Highest-risk components tested first; deferred features recorded as Partial

4. TEST CASES

Each test case has a unique identifier whose prefix shows its suite: TC-CORE (core library), TC-SVC (services), TC-SEC (security), TC-WEB (browser library) and TC-SYS (system). The full code of every case is in the repository, in the file named in the RTM. Tables 4.1 to 4.5 list every case with the requirements it verifies and its result on 4 November 2026.

4.1 Core Library

Table 4.1: Core library unit and performance test cases

ID	Test case	Requirements	Result
TC-CORE-001	Base45 matches RFC 9285 vectors (3 checks)	NFR-C1	Pass
TC-CORE-002	Base45 decodes RFC 9285 vector	NFR-C1	Pass
TC-CORE-003	Base45 round trip property	NFR-C1	Pass
TC-CORE-004	Invalid base45 raises value error (4 checks)	NFR-S5	Pass
TC-CORE-005	Base64url round trip property	NFR-C1	Pass
TC-CORE-010	Issued L2 credential verifies	FR-03, FR-16	Pass
TC-CORE-011	L1 credential without holder key verifies	FR-05	Pass
TC-CORE-012	L2 without holder key cannot be issued	FR-04	Pass
TC-CORE-013	Tampered name is rejected	FR-16, NFR-S5	Pass
TC-CORE-014	Any single bit flip is rejected	FR-16, NFR-S5	Pass
TC-CORE-015	Unknown issuer is rejected	NFR-S5	Pass
TC-CORE-016	Expired credential is rejected	FR-18	Pass
TC-CORE-017	Revoked credential is rejected	FR-18	Pass
TC-CORE-018	NIN is masked and full NIN is refused	NFR-S4	Pass
TC-CORE-019	Garbage never raises and is rejected	NFR-S5	Pass
TC-CORE-020	Any valid text round trips	FR-03, FR-16	Pass
TC-CORE-021	QR round trip	FR-16	Pass
TC-CORE-022	Credential with photo fits size budget	NFR-C1, FR-02	Pass
TC-CORE-023	Arbitrary text only raises invalid QR	NFR-S5	Pass
TC-CORE-024	Decompression bomb is rejected	NFR-S5	Pass
TC-CORE-025	Wrong prefix is rejected	NFR-S5	Pass
TC-CORE-030	Genuine wallet response verifies	FR-12, FR-17	Pass
TC-CORE-031	Replayed response fails on new challenge	FR-17, FR-23	Pass
TC-CORE-032	Response from another device fails	FR-17, FR-23	Pass
TC-CORE-033	Response is bound to credential ID	FR-17	Pass
TC-CORE-034	Challenges are unique 128-bit	FR-17	Pass
TC-CORE-035	Garbage response returns false (4 checks)	NFR-S5	Pass
TC-CORE-040	Signed list verifies	FR-09	Pass
TC-CORE-041	List with removed entry is rejected	FR-09, NFR-S5	Pass
TC-CORE-042	List from unknown key is rejected	FR-09, NFR-S5	Pass
TC-CORE-050	Intact chain verifies	FR-24, FR-25	Pass
TC-CORE-051	Editing any record is detected	FR-25, NFR-S6	Pass

ID	Test case	Requirements	Result
TC-CORE-052	Deleting a record is detected	FR-25, NFR-S6	Pass
TC-CORE-053	Reordering is detected	FR-25, NFR-S6	Pass
TC-CORE-060	Issuer key is encrypted at rest	NFR-S1	Pass
TC-CORE-061	Invalid payloads cannot be issued (5 checks)	FR-03, FR-04, NFR-S5	Pass
TC-CORE-062	Credential not yet valid is rejected	FR-18	Pass
TC-CORE-063	Signed but structurally invalid payload is malformed	NFR-S5	Pass
TC-CORE-070	Offline verification is far inside three seconds	NFR-P1	Pass
TC-CORE-071	Revocation list of 100k fits in two megabytes	NFR-C3, FR-09	Pass
TC-CORE-072	Signing throughput supports enrolment queues	NFR-P1	Pass

4.2 Services

Table 4.2: Service integration, performance and end-to-end test cases

ID	Test case	Requirements	Result
TC-SVC-001	Every service reports healthy	NFR-P3	Pass
TC-SVC-010	Login returns EdDSA token with role	FR-27	Pass
TC-SVC-011	Agent token lasts seven days for offline use	FR-14	Pass
TC-SVC-012	Five failed logins lock account until admin unlocks	NFR-S7, FR-29	Pass
TC-SVC-013	Missing or forged token is rejected (3 checks)	FR-28, NFR-S5	Pass
TC-SVC-014	Roles are enforced across services	FR-28	Pass
TC-SVC-015	Admin creates agent assigned to point	FR-29	Pass
TC-SVC-016	Login endpoint is rate limited	NFR-S7	Pass
TC-SVC-020	Enrolment uses NIMC and stores only masked NIN	FR-01, NFR-S4	Pass
TC-SVC-021	Issued L2 credential verifies with published keys	FR-03, FR-04, FR-08, FR-16	Pass
TC-SVC-022	Paper wallet and key rules	FR-05, FR-04	Pass
TC-SVC-023	Reissue is atomic and revokes old credential	FR-06, FR-09	Pass
TC-SVC-024	Revoked credential fails offline verification after sync	FR-07, FR-09, FR-18	Pass
TC-SVC-025	Trust refuses forged rollback or unauthenticated lists	FR-09, NFR-S5	Pass
TC-SVC-026	Revocation survives trust outage and republishes	FR-07, NFR-P3	Pass
TC-SVC-030	Batch upload is chained and idempotent	FR-24, FR-22	Pass
TC-SVC-031	Tampering in the database is detected	FR-25, NFR-S6	Pass
TC-SVC-032	Auditor filters records	FR-25	Pass
TC-SVC-033	Same credential at two points within 24h is flagged	FR-26	Pass
TC-SVC-040	Issuance completes within five seconds	NFR-P2, FR-03	Pass
TC-SVC-041	A full day of 2000 records uploads in one batch	NFR-C4, FR-24	Pass
TC-SVC-042	Large revocation list is served compressed	NFR-C3, FR-15	Pass
TC-SYS-001	Distribution day end to end	FR-01, FR-03, FR-04, FR-07, FR-15, FR-16, FR-17, FR-18, FR-22, FR-24, FR-25	Pass

4.3 Security

Table 4.3: Security test cases

ID	Test case	Requirements	Result
TC-SEC-001	Injection in login is rejected cleanly (6 checks)	NFR-S5, FR-27	Pass
TC-SEC-002	Injection in NIN is rejected by validation (8 checks)	NFR-S5, FR-01	Pass
TC-SEC-003	Injection in audit filters returns no rows	NFR-S5	Pass
TC-SEC-004	Forged tokens are rejected	FR-28, NFR-S5	Pass
TC-SEC-005	Expired token is rejected	FR-28, FR-14	Pass
TC-SEC-006	Agent cannot file records as someone else	FR-24, NFR-S6	Pass
TC-SEC-007	Oversized inputs are refused	NFR-C4, NFR-S5	Pass
TC-SEC-008	Security headers on every response	NFR-S5	Pass
TC-SEC-009	CORS allows only the web app origin	NFR-S5	Pass
TC-SEC-010	Errors never leak internals	NFR-S5	Pass
TC-SEC-011	API docs page has a strict content security policy	NFR-S5	Pass
TC-SEC-012	API docs can be switched off for production	NFR-S5	Pass

4.4 Browser Library

Table 4.4: Browser library test cases

ID	Test case	Requirements	Result
TC-WEB-001	Base45 RFC 9285 vector	NFR-C1	Pass
TC-WEB-002	Accepts a Python-issued L2 credential offline	FR-16	Pass
TC-WEB-003	Rejects a tampered credential	FR-16, NFR-S5	Pass
TC-WEB-004	Rejects revoked and expired credentials	FR-18	Pass
TC-WEB-005	Unknown issuer and garbage QR fail securely	NFR-S5	Pass
TC-WEB-006	Browser signs the same challenge response as Python (deterministic Ed25519)	FR-12, FR-17	Pass
TC-WEB-007	A different device key cannot answer the challenge	FR-23	Pass
TC-WEB-008	Verifies the Python-signed revocation list and rejects a forged one	FR-09, FR-15	Pass
TC-WEB-009	Device-to-device QR formats round-trip	FR-17	Pass
TC-WEB-010	Keeps a meaningful trailing space but drops line breaks	NFR-S5	Pass
TC-WEB-011	Recognises each device-to-device code type	NFR-U2	Pass
TC-WEB-012	Every vector still verifies after cleaning	FR-16	Pass
TC-WEB-013	100 full offline checks run far inside the 3 s budget	NFR-P1	Pass
TC-WEB-014	Device key is sealed by the PIN; a wrong PIN cannot open it	NFR-S2, FR-11	Pass
TC-WEB-015	Five wallet profiles with credentials fit easily in 5 MB	NFR-C2, FR-10	Pass
TC-WEB-016	100,000 revoked IDs are stored in well under 2 MB and round-trip exactly	NFR-C3	Pass

4.5 System Tests in the Browser

Table 4.5: Browser system test cases

ID	Test case	Requirements	Result
TC-SYS-002	Distribution day across three browser devices (officer, farmer, agent) (11 checks)	FR-04, FR-10, FR-11, FR-12, FR-13, FR-15, FR-16, FR-17, FR-19, FR-20, FR-22, FR-23, FR-24, FR-25, NFR-P3, NFR-U2	Pass
TC-SYS-003	Guided demo click-through, 16 steps against live services (16 checks)	FR-01, FR-03, FR-06, FR-07, FR-09, FR-14, FR-15, FR-16, FR-17, FR-18, FR-24, FR-25	Pass

The two browser system tests are scenario tests. TC-SYS-002 plays a full distribution day with three isolated browser contexts acting as the officer's laptop, the farmer's phone and the agent's phone: enrolment and issuance, saving the credential to the wallet, synchronising the verifier, cutting its network, then the offline check with challenge, PIN and face check, followed by a replay attempt, a tampered code and a revoked credential. TC-SYS-003 clicks through all sixteen steps of the Guided Demo against the live services.

5. TEST RESULTS

5.1 Results by Level

Table 5.1 summarises the results of the run on 4 November 2026.

Table 5.1: Results by test level

Level	Test cases	Checks run	Passed	Failed
Unit (core library)	38	50	38	0
Unit, performance (core)	3	3	3	0
Unit (browser library)	16	16	16	0
Integration (service APIs)	19	21	19	0
Integration, performance and capacity	3	3	3	0
Security (black-box)	12	24	12	0
System (API, end to end)	1	1	1	0
System (browser, Playwright)	2	27	2	0
Total	94	145	94	0

5.2 Code Coverage

Statement coverage was measured with coverage.py for Python and the V8 coverage provider for the browser library (Table 5.2). CI fails the build if Python coverage falls below 90%.

Table 5.2: Statement coverage by module

Code base	Module	Statements	Coverage
Core library	audit, challenge, encoding	82	100%
	credential	125	98%
	keys	20	95%
	revocation	39	92%
	qr	23	87%
	Total	290	97%
Services	shared (app, config, db, security)	124	98%
	issuer	180	97%
	audit	96	97%
	trust	64	97%
	auth	116	96%
	nimc, secrets	40	90%
	Total	620	97%
Browser library	vault.ts (PIN vault)		100%
	qrw.ts (verification)		90%
	api.ts, store.ts, router.tsx		0% (covered by system tests)

The uncovered Python lines are defensive branches that are hard to reach from outside, such as a key file missing at start-up. In the browser library, the security-critical code (verification and the PIN vault) is covered by unit tests; the network client, storage helper and router are thin glue code exercised by the Playwright system tests rather than by unit tests, which is why the overall browser figure (66%) is lower. Coverage measures what was executed, not what was checked, which is why it is used here alongside property-based tests and the requirement-level RTM rather than as a goal in itself.

5.3 Requirements Coverage

The generated RTM (QRW-SRS-01, Appendix A) reports 48 requirements, none failing and none uncovered. 43 requirements are implemented in full and 5 partially (FR-14, FR-27, NFR-S3, NFR-S8 and NFR-U1), with the missing part stated in the RTM. Nine requirements, mainly usability and data protection, cannot be fully checked by an automated assertion and were verified by inspection or demonstration, recorded in Table 5.3.

Table 5.3: Manual verifications

Requirement	Method and evidence
FR-19	Demo: Verifier face-check step (TC-SYS-002 screenshot 08)
FR-20	Demo: TC-SYS-002 screenshots 09-13
FR-21	Demo: Verifier demo clock +8 / +31 days
NFR-S3	Inspection: npm run phone serves HTTPS
NFR-S8	Inspection: data fields in credential and Issuer DB
NFR-U1	Inspection: Verifier verdict strings
NFR-U2	Inspection: TC-SYS-002 screenshots 09-10
NFR-U3	Demo: count taps in TC-SYS-002 flow (scan, scan answer, Yes)
NFR-U4	Inspection: styles.css

In addition, the candidate ran the 16-step Guided Demo by hand on his own Windows laptop on 3 November 2026 and every step passed. This acted as an informal acceptance test on the target development environment, separate from the automated TC-SYS-003.

5.4 Performance and Capacity

The performance tests measure as well as assert, so the actual figures are recorded (Table 5.4).

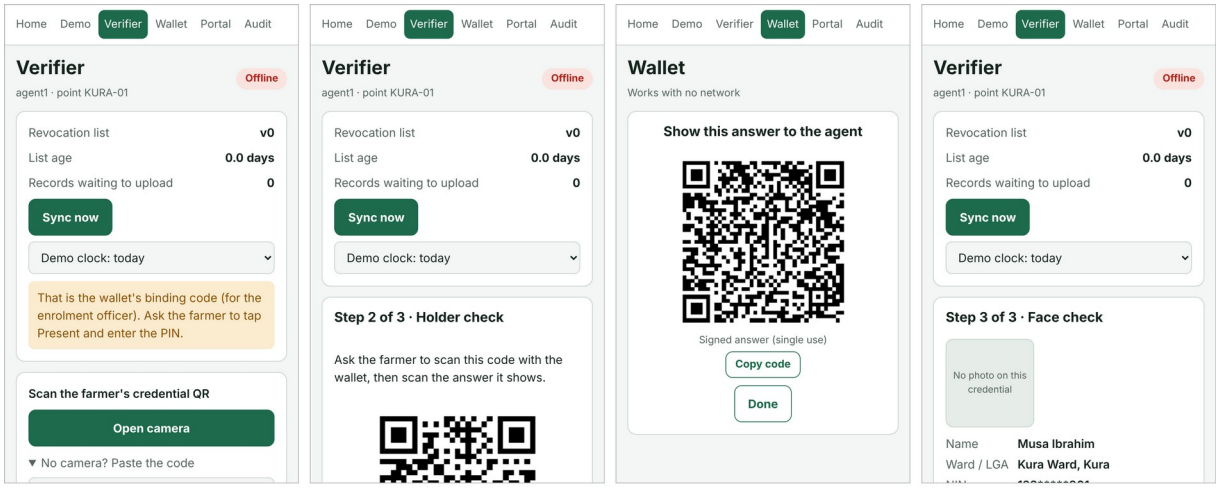
Table 5.4: Measured performance against budgets

Test	Requirement	Budget	Measured
TC-CORE-070	NFR-P1 full offline check (Python)	3,000 ms	0.32 ms
TC-WEB-013	NFR-P1 full offline check (browser library)	3,000 ms	4.3 ms
TC-SVC-040	NFR-P2 issuance through the API	5,000 ms	8 ms
TC-CORE-071	NFR-C3 100,000-entry revocation list	2 MB	2.40 MB plain; 0.94 MB compressed
TC-WEB-016	NFR-C3 same list in the phone cache	2 MB	1.41 MB
TC-WEB-015	NFR-C2 five wallet profiles	5 MB	3.4 KB
TC-SVC-041	NFR-C4 one day of 2,000 records	One request	Accepted; chain intact

The measurements were taken on a test machine, not a low-end phone. A budget-class Android phone may be ten to fifty times slower, which would still leave verification under a quarter of a second against a three-second budget.

5.5 System Test Results

Both browser system tests passed: TC-SYS-002 with 11 of 11 checks and TC-SYS-003 with 16 of 16 steps. Figure 5.1 shows four screens from TC-SYS-002 during the offline check; the verdict screens for genuine, replayed, edited and revoked credentials are in QRW-SEC-03, Figure 5.1.



(a) Synced, now offline: ready to scan (b) Verifier shows challenge (c) Wallet answers after PIN (d) Agent checks the face

Figure 5.1: Offline verification during TC-SYS-002

5.6 Static Analysis, Security Scans and Continuous Integration

Table 5.5: Static and dynamic analysis results

Check	Result
Flake8 (Python lint)	Clean
TypeScript strict type-check	Clean
Bandit (Python SAST)	One low finding triaged as a false positive (B105, OAuth2 token type); one fixed (subprocess with shell=True in a tool)
pip-audit	No known vulnerabilities
npm audit (shipped packages)	react-router-dom advisories found; dependency removed; now 0
OWASP ZAP 2.16.1	Before: 0 High, 1 Medium. After fix: 0 High, 0 Medium, 1 Low accepted

The pipeline runs on GitHub were all green at the time of capture, with all four jobs passing on each (Figure 5.2).

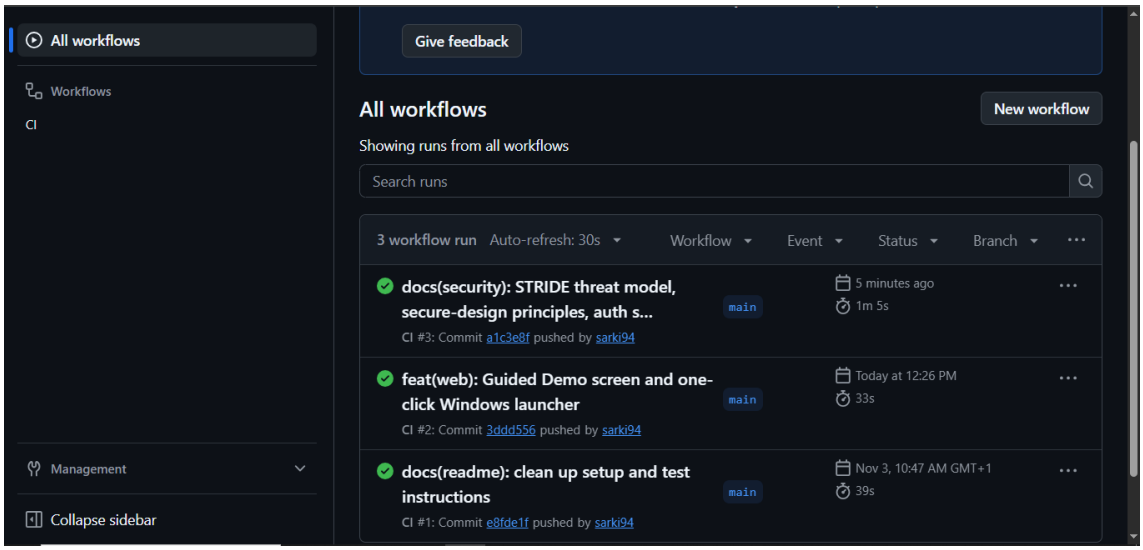


Figure 5.2: GitHub Actions: continuous integration runs, all jobs passing

6. DEFECTS FOUND

Tests are only useful if they find problems. Table 6.1 lists the defects the tests and scans found during development, how each was found and fixed, and the test that now guards against its return.

Table 6.1: Defect log

ID	Found by	Defect	Severity	Fix and regression test
DEF-01	Manual test by the candidate (3 Nov)	The scanner trimmed input, removing trailing spaces that are valid Base45 characters, so some genuine codes were rejected as "not a valid identity code"	High	Keep meaningful spaces, drop only line breaks; clearer messages per code type (4888af3). TC-WEB-010 to 012
DEF-02	TC-CORE-071 (performance)	A 100,000-entry revocation list was 2.40 MB, over the 2 MB budget	Medium	Compression in transit and a packed phone cache (7264652). TC-CORE-071, TC-SVC-042, TC-WEB-016
DEF-03	npm audit	react-router-dom had open-redirect and cross-site scripting advisories	High	Dependency removed; internal router (6758cc8). CI audit gate
DEF-04	OWASP ZAP	API documentation pages had no Content-Security-Policy	Medium	Hash-based policy; docs switch for production (7c44f43). TC-SEC-011, 012
DEF-05	OWASP ZAP re-scan	First fix still allowed inline styles	Medium	Inline styles removed; checked in a browser (7c44f43). TC-SEC-011
DEF-06	Bandit	A developer tool ran subprocess with shell=True	Low	Command resolved without a shell (7264652). Bandit in CI
DEF-07	Candidate's walkthrough (3 Nov)	Pasting the wrong kind of code into the Verifier gave an unhelpful error	Low	Code type recognised and explained (4888af3). TC-WEB-011

The design also changed as a result of reviewing it before code existed: checking the issuance sequence diagram showed that a re-issue had to revoke the old credential in the same transaction as it saved the new one (decision D8), which is now verified by TC-SVC-023.

7. EVALUATION AND CONCLUSION

7.1 Evaluation

The evidence supports the claim that the prototype meets its requirements, with the stated partial exceptions. The strongest evidence is for the parts that matter most: the verification logic is tested by examples, by properties over generated inputs, and against an independent implementation in another language, and the whole flow is tested offline in real browsers. The tests were not written after the fact to confirm working code; the core was written test-first, and the tests, scans and the candidate's own manual testing found seven real defects, each now guarded by a regression test.

7.2 Limitations

- No field test on a low-end Android phone in Kura has been done yet; performance figures come from a test machine.
- The Playwright system tests are run by hand, not in CI, because they need the services and a built web app running together.
- There is no load test of many simultaneous synchronisations.
- Usability was checked by inspection and demonstration, not by testing with farmers or agents.

7.3 Conclusion

94 test cases at five levels, all passing, with 97% statement coverage of the Python code, full requirement traceability, continuous integration on every push and documented security scans give a high level of confidence in the prototype for an examination system. The most valuable next step in testing is a field trial on the phones and network conditions the system is designed for.